Physical AI 応用編1 2026

松尾・岩澤研究室
MATSUO-IWASAWA LAB UTOKYO

第5回

許諾なく撮影や第三者
への開示を禁止します

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

今回の講義の準備

1. 前回（第4回）の演習用Docker Imageを使います
   Imageのpullがまだの方は以下を実行しておいてください

```
docker pull airobotbook/ros2-desktop-ai-robot-book-humble:latest  # 第４回講義で実行済みの人は不要
```

- 今回は可視化ツール **rerun** を使うため，コンテナに**ポートを2つ追加**して作り直します
  - **`-p 9090:9090 -p 9876:9876`** を追加
  - 新しいコンテナ名は以前と別の名前にする（例：**`pai_ros2`**）
- 手順の詳細は「演習環境の準備」（後述）で説明
- 作業はこれまでどおりブラウザで **`http://localhost:6080/`** を開く

2. Weights & Biases（wandb）のアカウントを作っておくと，学習の経過をグラフで見られる
   ログインしなくても学習はできる（WANDB_MODE=offline）
   ログインする場合は学習の前に wandb の更新（1行）が必要（学習のページで説明）

※ VNC接続時に「**変換エンジンプログラムの起動に失敗しました…（Mozc）**」と出ても**閉じてOK**

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Agenda

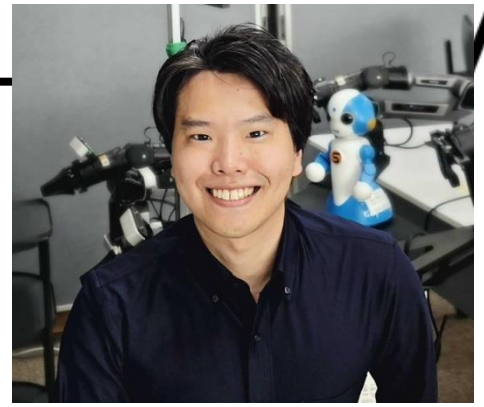
1. 導入
2. 第4回 ROS 2入門の復習
3. ROSによるデータ収集（TurtleSim）
4. LeRobotを用いた模倣学習
5. まとめ

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

1. 導入

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

講師自己紹介



長野 匡隼 (Masatoshi Nagano)

- 京都大学 大学院情報学研究科 記号創発システム分野（谷口研）助教

Home page :
https://sites.google.com/view/nagano-lab/profile

略歴

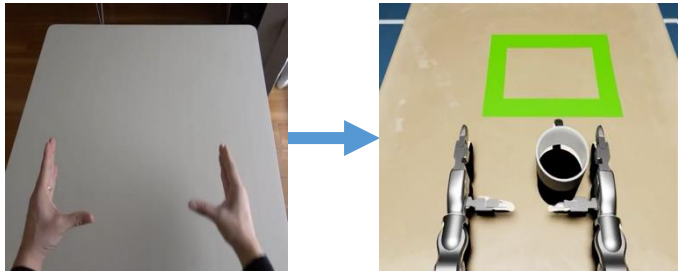
- ～2023：学士～博士（電気通信大学）
  - JAXA 非常勤職員，本庄国際奨学財団 奨学生，学振DC2
- 2023～2024：研究員（NTT R&D @横須賀）
- 2024～2025：特任研究員（電気通信大学）
- 2025～：助教（京都大学）& 客員研究員（電気通信大学）

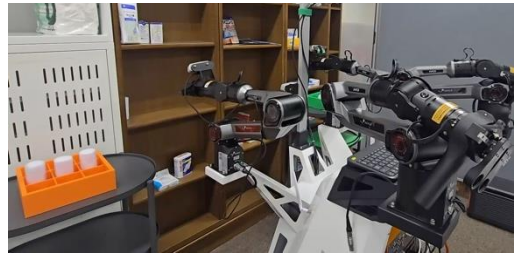


記事：https://note.com/chai_plus/n/ne6c33c784721

研究分野：**時系列データ解析・模倣学習・ロボットシステム**

- 人間と共生できる適応的なロボットやシステムの構築
- ロボットや学習理論から人間の認知メカニズムを構成論的に理解
- キーワード
  - 時系列情報の分節化（マルチモーダル時系列データ解析）
  - 見真似学習（ロボティクス & CV & 模倣学習）
  - ロボットシステム（ロボティクス & 標準化）





©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

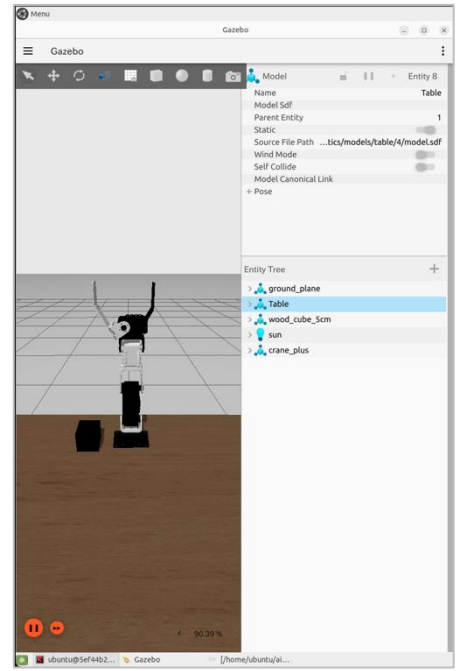
参考資料

- 「入門モダンLinux」（Michael Hausenblas 著，竹内覚・大岩尚弘訳）
  - Linuxの基本知識
- ROS公式ページ：https://www.ros.org/
  - Docs：https://docs.ros.org/
  - ROS 2 Humble Tutorials：https://docs.ros.org/en/humble/Tutorials.html
- 「ROS 2とPythonで作って学ぶAIロボット入門 改訂第2版」
  （出村公成・萩原良信・升谷保博・タン ジェフリー トゥ チュアン 著）
  - ROS2の入門本
  - 実装とともにマニピュレーションやナビゲーションの基礎を学べる
  - 演習環境として活用：https://github.com/AI-Robot-Book-Humble/docker-ros2-desktop-ai-robot-book-humble
  - https://github.com/Tiryoh/docker-ros2-desktop-vnc
- LeRobot：https://github.com/huggingface/lerobot
- 演習用リポジトリ
  - https://github.com/matsuolab/pai_training
  - https://github.com/matsuolab/physicalai_teleop

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

今回の概要

- **目標**
  ロボットのデータ・モデルを「収集，確認（変換），学習，デプロイ，評価」の一連のプロセスを実装できるようになる



- 前半：ROS 2でのデータ収集と模倣学習の準備
  - 第4回 ROS 2入門の復習
  - ロボット学習システムにおけるROSの位置付け
  - rosbag2によるデータの記録・再生（turtlesim）
  - LeRobotの環境構築とデータセットの可視化
- 後半：実践演習
  - Gazebo内のCRANE+ V2をテレオペしてデータ収集
  - rosbagからLeRobotDataset への変換・可視化
  - physicalAI技術による模倣学習
  - 学習した方策をデプロイ・デバッグ



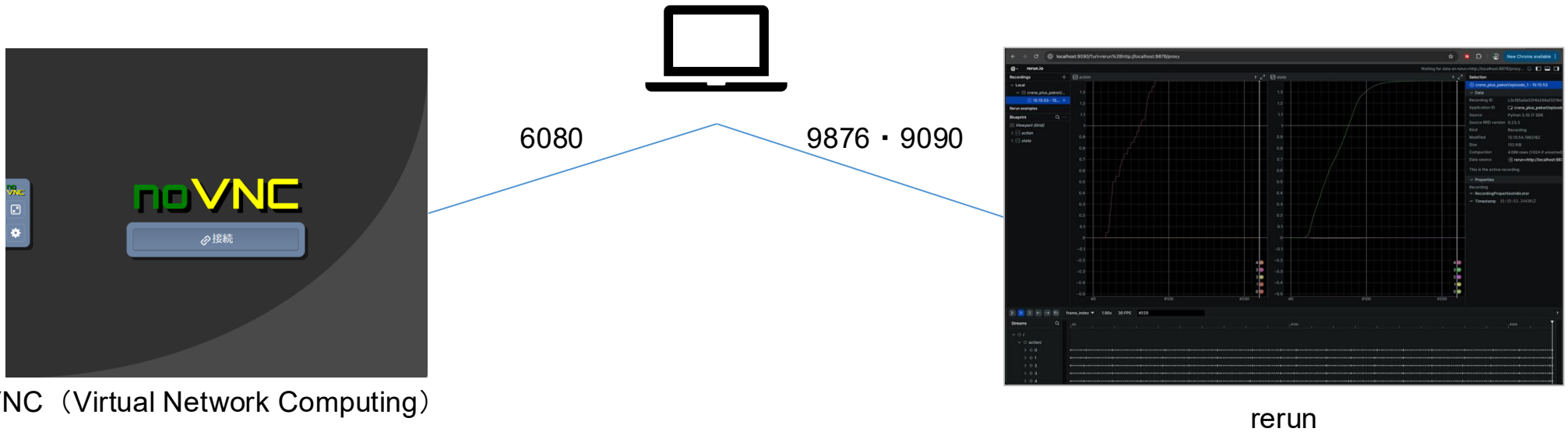
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

演習環境の準備（1/3）：ポートマッピングの追加

- 今回の演習では **rerun** というビジュアライザを使うため，コンテナに新たにポートマッピングを追加
  - **`9090`，`9876`**：rerunのWebビューア用．手元PCのブラウザから可視化結果を見る

⚠ **VSCode / Cursor などの Dev Container 拡張機能でコンテナに接続する場合は，以下の作業は不要（やっても良い）**

- 拡張機能がコンテナ内で公開されたポートを自動的に検出し，ホスト側へ転送（ポートフォワーディング）してくれるため，`-p` オプションを指定してコンテナを作り直す必要がない
- VSCode左下のマーク →「Attach to Running Container…」→ 演習用コンテナを選択して接続し，あとは **`http://localhost:6080/`** で作業すればOK



VNC（Virtual Network Computing）

rerun

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

演習環境の準備（2/3）：既存コンテナの停止

1. イメージの取得（前回講義で実行済みの人は不要）

```
docker pull airobotbook/ros2-desktop-ai-robot-book-humble:latest
```

2. まず既存の演習用コンテナを止める

```
docker ps                      # 走っている演習用コンテナ名を確認
docker stop <CONTAINER_NAME> # コンテナ名は ros2_docker_practice や ai_robot_book
```

- 削除したければ docker rm <CONTAINER_NAME>

3. 新たなコンテナ作成
   - コンテナ起動スクリプトにポートマッピングを2つ追加する（次ページ）
   - -p 9090:9090 -p 9876:9876
   - 新たに作るコンテナの名前を以前と違うものに設定する（例：pai_ros2）

⚠ **最初に Docker Desktop を起動：`open -a Docker` → `docker info` でエラーが出なければOK**
起動していないと `Cannot connect to the Docker daemon ... Is the docker daemon running?` になる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

演習環境の準備（3/3）：起動スクリプトの作成

3. 新たなコンテナ作成（4回を参考に作成「第4回：docker run -p 6080:80 --shm-size=512m ... 」）
   - 作業フォルダを作り，その中でスクリプトを作って実行する

```
cd ~/Desktop && mkdir -p 261001_pai_ros2_no2 && cd 261001_pai_ros2_no2 # どこでもよい
vim run_pai_container.sh     # nano でもよい．下の内容を書いて保存
cat run_pai_container.sh     # 中身を確認
chmod +x run_pai_container.sh && ./run_pai_container.sh   # 2回目以降は docker start pai_ros2
```

   - -p 9090:9090 -p 9876:9876
   - 新たに作るコンテナの名前を以前と違うものに設定する（例：pai_ros2）

※ macOSでうまくいかないor警告を消すときは「--platform linux/amd64 ¥」を-pの前に

```
#!/bin/bash
docker run -d --name pai_ros2 \
  -p 6080:80 --shm-size=512m \
  -p 9090:9090 -p 9876:9876 \
  -v "$(pwd)":/workspace \
  airobotbook/ros2-desktop-ai-robot-book-
humble:latest
```

run_pai_container.shの中身

- -d：裏で起動（端末が空く）
- --name：コンテナの名前
- -p：ポートマッピング
- --shm-size：共有メモリ（今回512m）
- -v：マウント用（成果物を手元のLocal PCに残せる）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】演習環境の準備：起動スクリプトの変更の場合

run_ai_robot_container.sh の場合
（システムエンジニアリング第2回講義受講者）

```
#!/bin/bash
docker run \
  -e RESOLUTION=1920x1080 \
  --name pai_ros2 \
  -p 15900:5900 \
  -p 13389:3389 \
  -p 9090:9090 \      # 追加
  -p 9876:9876 \      # 追加
  -p 6080:80 \
  --shm-size=512m \
  --privileged \
  airobotbook/ros2-desktop-ai-robot-book-humble
```

arb-run.bash の場合
（docker-ros2-desktop-ai-robot-book-humble）

```
#!/bin/bash
MY_IMAGE=ros2-desktop-ai-robot-book-humble:mine
docker run \
  -e http_proxy=${http_proxy} \
  ...（プロキシ・音声関連の設定は既存のまま）
  -e RESOLUTION=1920x1080 \
  --name pai_ros2 \
  -p 15900:5900 \
  -p 13389:3389 \
  -p 9090:9090 \      # 追加
  -p 9876:9876 \      # 追加
  -p 6080:80 \
  --shm-size=512m \
  --privileged \
  $* \
  $MY_IMAGE
```

- 変更後，スクリプトを実行してコンテナを起動

```
./run_ai_robot_container.sh   # or ./arb-run.bash
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

2. 第4回 ROS 2入門の復習

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

ROSの概要

- **ROS（Robot Operating System）**：ロボットのソフトウェアコンポーネントを開発・統合するためのフレームワーク
  - 環境の再現性の向上
  - 様々なパッケージを使用でき,簡単にロボットを操作可能

Plumbing（通信基盤）	Tools（ツール群）
容易に分散コンピューティングシステムを構築できるように設計された通信基盤	開発・起動・監視・可視化・ログ取得などのツール群 例：**`ros2 topic`, `rviz2`, `rqt_graph`**
Capabilities（機能群）	**Ecosystem（エコシステム）**
移動・マニピュレーション・知覚などの機能をロボットに実装するためのライブラリ 例：**`navigation2`, `moveit2`**	世界中の開発者が公開する多くのROSパッケージを利用可能

- **ROS 2**：ROS 1の課題を解決し，マルチプラットフォーム対応・リアルタイム制御・セキュリティ強化を実現した次世代版

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

ROS 2の4つの通信方式

ROS 2では，ノード同士が以下の通信方式でデータをやり取りする

名前	通信方式	用途
トピック通信	一方向・非同期	複数のノードに同じデータを一方的に送りたい場合に使用 例：センサデータの配信・受信
サービス通信	双方向・同期	リクエストを送って処理がすぐに終わるようなタスクに使用 例：デバイスの起動・終了，モード切り替え
アクション通信	双方向・非同期	処理に時間がかかり，途中経過（フィードバック）も取得したいときに使用 例：ナビゲーション，複雑な動作
パラメータ通信	双方向・同期	実行中にノードのパラメータ設定を取得・変更・監視したいときに使用 例：PIDゲインの変更

今回のデータ収集（rosbag2）では，主に**トピック通信**のデータを記録する

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【付録】頻出ROS 2コマンドまとめ（1/2）：全般

コマンド	内容（実行場所）
`ros2 pkg create PACKAGE_NAME --build-type ament_python`	パッケージの作成（~/WORKSPACE/src）
`rosdep install --from-paths src --ignore-src -r -y`	src/以下のパッケージの依存関係を解決・インストール（~/WORKSPACE）
`colcon build --symlink-install --packages-select PACKAGE_NAME`	指定パッケージのみビルド．--symlink-install でPython変更時の再ビルドが不要に
`source ~/WORKSPACE/install/setup.bash`	ビルドしたパッケージをROS 2環境に認識させる **（~/.bashrcに書くと便利）**
`ros2 run PACKAGE_NAME NODE_NAME`	指定したノードを起動（最も基本的なコマンド）
`ros2 launch PACKAGE_NAME LAUNCH_FILE`	launchファイルで複数のノードをまとめて起動
`ros2 node list` / `ros2 pkg list`	実行中のノード一覧 ／ インストール済みパッケージ一覧
`rqt_graph`	ノードとトピックのつながりを図で見る
`ros2 run rqt_image_view rqt_image_view` / `rqt`	カメラの映像を見る ／ rqt：各種ツールをまとめて呼び出せる入れ物

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【付録】頻出ROS 2コマンドまとめ（2/2）：通信関係

トピック通信

- **`ros2 topic list`**：アクティブなトピック一覧
- **`ros2 topic echo TOPIC`**：流れているメッセージを表示
- **`ros2 topic type TOPIC`**：メッセージ型を表示
- **`ros2 topic hz TOPIC`**：メッセージの周期（Hz）を表示

サービス通信

- **`ros2 service list`**：利用可能なサービス一覧
- **`ros2 service call NAME TYPE REQUEST`**：サービスを呼び出す

アクション通信

- **`ros2 action list`**：利用可能なアクション一覧
- **`ros2 action send_goal NAME TYPE GOAL`**：ゴールを送信

パラメータ通信

- **`ros2 param list`**：各ノードのパラメーター一覧
- **`ros2 param get NODE PARAM`**：値を取得
- **`ros2 param set NODE PARAM VALUE`**：値を設定

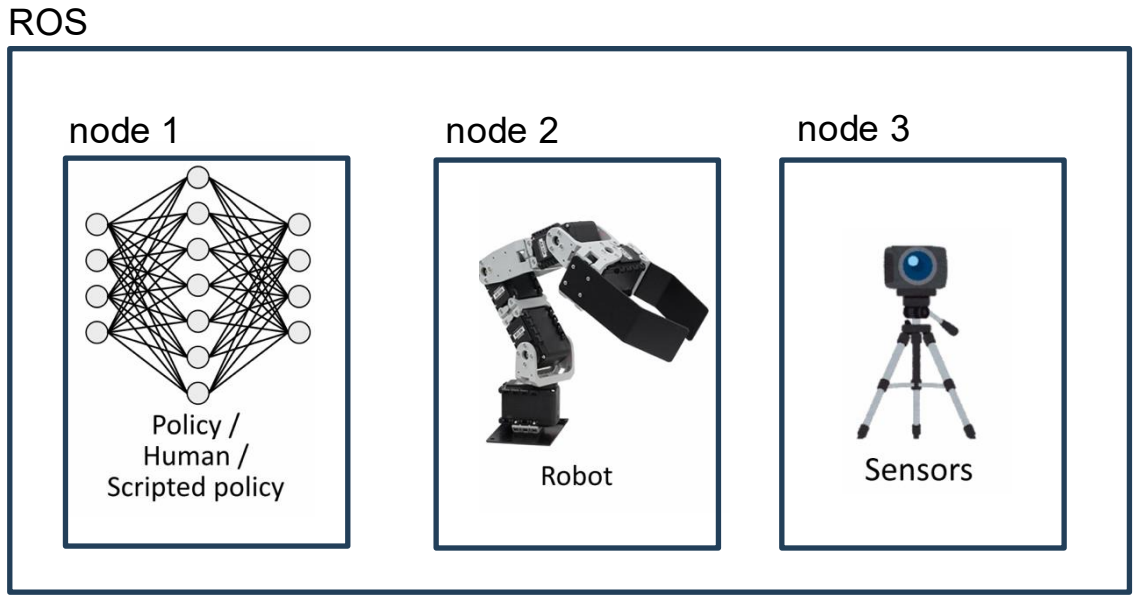
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

3. ROSによるデータ収集（TurtleSim）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

ロボット学習システムにおけるROSの位置付け

- ROSは主に**実機ロボットとの通信インターフェース**としての役割を担う
- 実機ロボットや各種センサ（RGB-Dカメラ，力覚センサ，触覚センサ）とのインターフェースとしてROSを使うと，**データのやり取りが楽**
  - ノードが独立しているので，**開発・復旧・デバッグが容易**
- 多くのロボット・センサがROSに対応しており，メーカーがROSのwrapperを公開している
  - https://github.com/rt-net/crane_plus
  - https://github.com/IntelRealSense/realsense-ros



ロボットシステム例

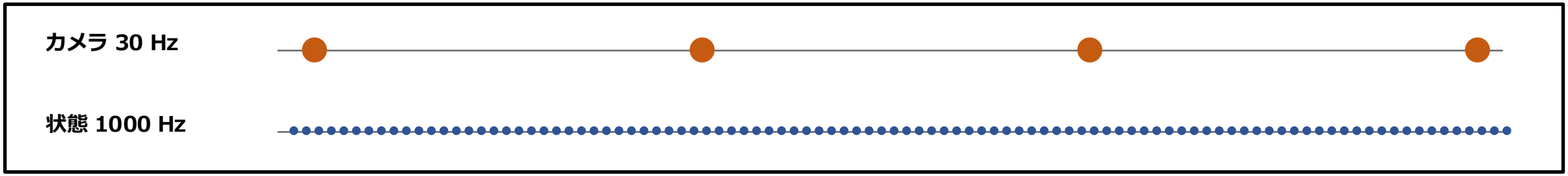
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】ROSを使わなくても単純なシステムは作れるが…（1/2）

- **同期的（ブロッキング）な処理**になってしまう.
- 例えば，方策 π(a | o, s) で行動を決めるとして，単純に次のように書くと…

```
for timestep in range(max_timestep):
    img = get_rgb_image()          # 30 Hz で更新されるカメラ画像を取得
    state = get_robot_state()      # 1000 Hz で更新される関節角度などを取得
    action = policy(img, state)    # モデルの推論
    send_robot_command(action)     # ロボットに指令を送る
```

- 次のループのために，**get_rgb_image**や**get_robot_state**が返ってくるまで待つ必要がある
  - **最も遅いセンサ（この例では30 Hzのカメラ）に全体の周波数が縛られ**，それ以上速い周期で行動できない
- **画像と関節角の両方がそろわないと行動を出せない**



周波数のイメージ（ループの周期はカメラの到着（●）待ちで決まる）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】ROSを使わなくても単純なシステムは作れるが…（2/2）

- **デバイスごとにドライバ**（デバイスを動かすためのソフトウェア）が必要
  - USBカメラならOpenCVのAPI，関節状態ならシリアル通信，モータ指令ならCANバス…
    - それぞれ低レベルAPI／SDKを直接叩く
  - これらを全て自分でwrapし，エラー処理・リトライ・再接続などのロジックを書き，複数スレッドから同時に呼ぶときの**排他制御**も考慮



複数スレッドから同時に呼ぶ

- ハードウェア：「撮影して」「止めて」が同時に届き，状態がギクシャク・フリーズ
- 共有データ：書き込み途中で上書きされリストが壊れる（レースコンディション）
- いつ・どのスレッドが壊したか分からず，デバッグが非常に困難



ROSを使う

- ノード間通信（publish／subscribe）の仕組みが，内部で**非同期処理や排他制御を吸収**してくれる
- ユーザは自分でロックをかけたり，並列アクセスの競合を気にしたりせずに済む

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

ROSによるデータ収集：rosbag2

- ロボット学習，特に模倣学習では，**人間がロボットを遠隔操作する際のデータを記録**することが必要
- ROSベースのロボットシステムには，**rosbag2** というデータ記録・再生用の便利なツールがある

rosbag2でできること

1	**全トピックにpublishされているデータの保存**
2	**保存されたデータの再生**
3	**timestampを利用した，データの任意周期でのサンプリング**

※ bagは「袋」の意味．rosbagは，ROS関連のデータを入れておく袋のようなもの

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

演習：Turtlesimでrosbag記録・再生（1/3）

- turtlesimを起動し，遠隔操作を介してデータをrosbagで記録・再生する

1. コンテナを起動
2. Terminatorを開く（3つに分割）
   - Ctrl+Shift+E: 縦分割， Ctrl+Shift+O: 横分割, Ctrl+Shift+W: 削除
3. （ターミナルA）ノード turtlesim_node を起動

```
ros2 run turtlesim turtlesim_node
```

4. （ターミナルB）遠隔操作ノードを起動

```
ros2 run turtlesim turtle_teleop_key
```

5. （ターミナルC）rosbagファイルを保存するディレクトリを作成

```
cd ~
mkdir -p bag_files && cd bag_files
```



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

演習：Turtlesimでrosbag記録・再生（2/3）

6. （ターミナルC）記録開始．例ではファイル名は turtle_test（なんでもOK）

```
ros2 bag record -o turtle_test -a
```

7. ターミナルBをクリックしてから矢印キーで，例えば円を描くように亀を遠隔操作
   終わったらターミナルCに戻り，Ctrl+C で記録停止
8. （ターミナルC）turtle_test という名前でデータが保存されていることを確認

```
ubuntu@5ef44b204faa:~/bag_files$ tree turtle_test/
turtle_test/
├── metadata.yaml
└── turtle_test_0.db3

0 directories, 2 files
```

9. （ターミナルC）turtle_test を再生

```
ros2 bag play turtle_test/
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

演習：Turtlesimでrosbag記録・再生（3/3）

- **記録した軌道と同じ動き**をすることが確認
- 初期位置を揃えて確認したい場合
  – 一度ターミナルAの turtlesim_node を Ctrl+C で落としてから再度立ち上げ，9を実行

- 余力があれば
  - どんなデータや設定が保存されているか確認しよう！

```
ros2 bag info turtle_test/
```

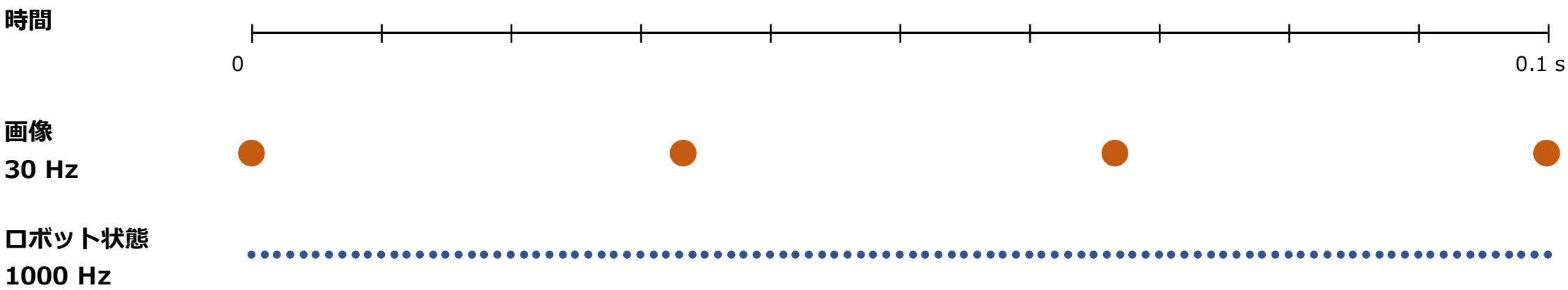
  - 記録されたトピックとメッセージ数も確認



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】rosbag2の記録データは非同期

- rosbag2で収集されるトピックデータは**非同期**であり，そのまま学習に使えるものではない
- モダリティによってデータの周波数（密度）が異なる



画像は約0.033秒ごとに1つ，ロボット状態は0.001秒ごとに1つ（0.1秒間の例）

- 強化学習・模倣学習では，方策は主に **π = p(a | o)** で定義され，観測 **o = [o_image, o_state]** の各要素の**時刻が同期**されている必要がある
  - 本来非同期な観測をネイティブに非同期で受け取れるアーキテクチャが必要では，という議論もあるが一旦置いておく
- 学習に使うには，timestampを用いて全データを**特定の周期でサンプリングして同期**させる処理が必要
  → 後半の演習（**rosbag2lerobot変換**）で実施

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】rosbag2コマンド一覧

トピックの記録

```
ros2 bag record TOPIC_NAME
ros2 bag record -a                    # 全トピックの記録
ros2 bag record -o BAG_FILE_NAME -a # -o オプションでファイル名の指定
ros2 bag record -a --compression-mode file --compression-format zstd  # zstdで圧縮して保存
```

トピックの再生

```
ros2 bag play BAG_FILE_NAME
```

記録したbagファイルの情報表示

```
ros2 bag info BAG_FILE_NAME
```

- **record**：記録　／　**play**：再生　／　**info**：記録時間・トピック・メッセージ数などを確認

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】なぜturtlesimで練習したのか

- 人がロボットを操作したデータ（**指令と状態**）を記録する
- ROSでtopicを保存・再生できれば良い → rosbag を使う
- 標準的なシミュレーションであるturtlesimで **記録** → **確認** → **再生** の流れを体験する
- CRANE+ V2などのロボットだとしても収集は同じ

	turtlesim（練習）	CRANE+ V2（後半の演習）	模倣学習での役割
テレオペ	`turtle_teleop_key`	`teleop_keyboard`	**人間のデモ**
指令	`/turtle1/cmd_vel`	`/crane_plus_arm_controller/joint_trajectory`	**action**
状態	`/turtle1/pose`	`/joint_states`	**state（観測）**
記録	**`ros2 bag record -a`**	**`ros2 bag record -a`**	1 bag＝1エピソード

cmd_vel が後半の **action**，**pose** が **state** にあたる（**ros2 bag info** で両方の記録を確認）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】学び方やエラーに対して

わからなくても、とりあえず手を動かす

- 最初は何もわからない
- 全部理解してから動かそうとするのはコスパが悪い（ほぼ不可能）

一回動かしてみて、気になったことを調べる

- 実行しているコマンドの意味，中で何が起きているか，どんな通信が行われているか
- 「動いた！」「なんで動いてるんだ？どういう仕組み？」

エラーは学びのチャンス

- エラーに遭遇したら喜ぶ．強くなるチャンス
- 対処の流れは次ページ

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】エラーに遭遇したときの進め方

0	**理解しようとする**	生成AI（Claude Code・Cursor など）に作成・修正もらった場合も， **何のエラーで，どう解決したか**を必ず確認する
1	**エラー文を全部読む**	どの辺が問題になっていそうか当たりをつけてみる どの辺が問題だったのか確認する
2	**検索する**	エラー文が複雑なときは生成AIなどにコピペし，解決した先人を探す ROSはユーザが多く，答えが見つかる場合が多い
3	**LLMや詳しい人に聞く**	実行したコマンド・環境・エラー全文など，コンテキストを十分に与える 解決策が複数あるうちのどれだったかも比べる
4	**解決**	0～4を繰り返すうちに**自分が学習される** 1の時点で原因が察することや、0で事前に計画できるようになってくる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

4. LeRobotを用いた模倣学習

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

LeRobotを用いた模倣学習：データセットの可視化

公開データセットから可視化を体験しよう

- 収集したデータセットを検証する上で，可視化は非常に重要
- LeRobotのデータ可視化スクリプトでは，rerun というモダンな可視化ツールが使われている
- 今回は，Hugging Face Hub に lerobot/pusht という repo-id（データセット名のようなもの）で登録されている公開データセットを手元で可視化する

- 可視化の流れ

1. **環境構築（pixi）**
2. **公開データセットを取得**
3. **rerunで可視化**

- 模倣学習には，**LeRobot**（Hugging Face）というオープンソースのフレームワークを利用
  - Hugging Faceでは，「データセット」「モデルの重み（ウェイト）」「コード」が公開
  - https://github.com/huggingface/lerobot

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

1. LeRobotの環境構築

- 前回紹介したパッケージマネージャ **pixi** で環境構築し，LeRobotを **git submodule** にする
- https://github.com/matsuolab/pai_training のREADMEに従ってセットアップ

```
cd ~ && git clone --recursive https://github.com/matsuolab/pai_training
cd pai_training
```

- pixi をインストールし，**`pixi install`** で簡単に依存解決

```
curl -fsSL https://pixi.sh/install.sh | bash
echo 'eval "$(pixi completion --shell bash)"' >> ~/.bashrc
source ~/.bashrc
```

```
pixi install
```

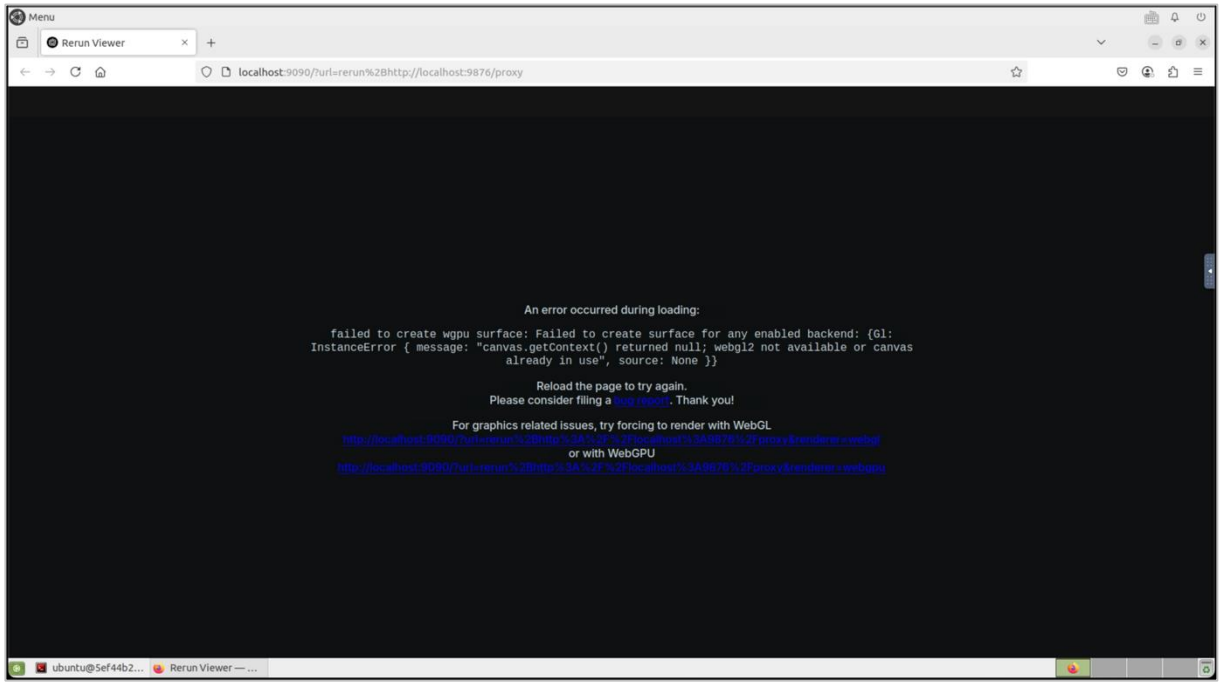
- pixiの最低限の使い方はREADMEを参照
- 詳細は公式documentationへ

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

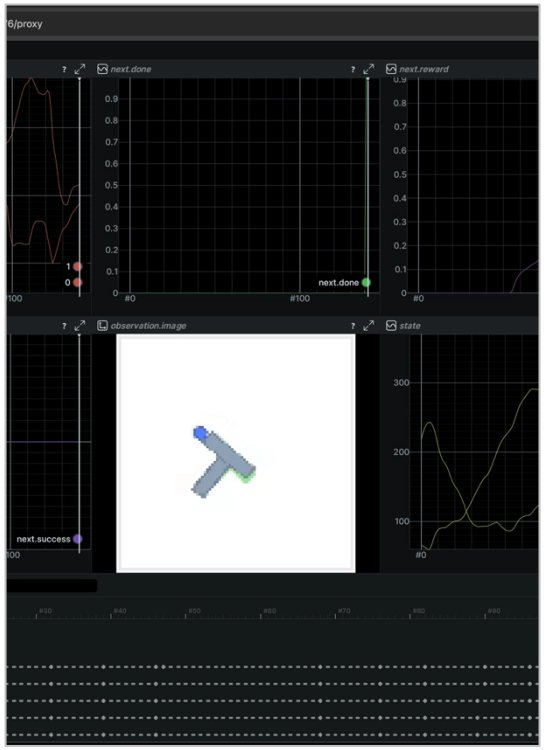
2-3.公開データセットの取得・可視化

```
pixi run python scripts/visualize_dataset.py \
    --repo-id lerobot/pusht --episode-index 0 --mode distant
```

※ **HTTP Error** や **Too many requests** で止まることがあるが，数秒空けて何回か試すとうまくいく



Firefoxがポップアップしたら，アクセスしているURLをコピー
手元PCのWebブラウザに貼り付けてアクセス



左下の ▶ ボタン（or スペースキー）で再生

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO



休憩

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】可視化のポイント

- 可視化による**データの性質の直感的な理解・データの検証**は重要
  - 画像・状態・行動が時間的に対応しているか
  - 値の飛びや欠損，想定外のスケールがないか
  - エピソードの開始・終了が意図どおりか

→ 意図しないデータや外れ値があると，**モデルが誤った学習をしてしまう**

⚠ 9090ポートと9876ポートを「演習環境の準備」に従ってポートマッピングしていれば，ローカルブラウザで可視化できるはず

→ **どうしても難しい場合，可視化はスキップして大丈夫**

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

LeRobotを用いた模倣学習：実践演習の全体像

- Gazebo内の **CRANE+ V2** マニピュレータを遠隔操作（テレオペ）してデータを集め，集めたデータで模倣学習を回す



工程	使うもの	入力 → 出力
データ収集	physicalai_teleop，ros2 bag	テレオペ操作 → rosbag（1エピソード＝1 bag）
変換・可視化	pai_training（pixi run convert），rerun	rosbag → LeRobotDataset
学習	pai_training（pixi run train），wandb	LeRobotDataset → 学習済みACT
デプロイ・デバッグ	pai_training（pixi run infer / debug-policy）	/joint_states → 関節指令トピック

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

テレオペ用パッケージ取得

- physicalai_teleop パッケージの README.md に従う
  （いつも通りの新しいパッケージのセットアップ）

```
mkdir -p ~/pai_ws/src
cd ~/pai_ws/src && git clone https://github.com/matsuolab/physicalai_teleop.git

# 依存関係解決．ビルドの前に必要なパッケージをインストール
cd ~/pai_ws && rosdep install --from-paths src --ignore-src -r -y

# ビルド．--symlink-install で install/ へのコピーの代わりにシンボリックリンク
colcon build --symlink-install --packages-select physicalai_teleop

# 環境変数を設定．physicalai_teleop パッケージが認識されるようになる
source install/setup.bash
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Gazebo & CRANE+ V2 とテレオペノードの起動

Gazebo & CRANE+ V2 起動

```
ros2 launch crane_plus_gazebo crane_plus_with_table.launch.py
```

```
ubuntu@5ef44b204faa:~/pai_ws$ ros2 run physicalai_teleop teleop_keyboard

Teleop Crane+ V2 (Direct Joint Control):
---------------------------------------
Q/E: Select joint (1-4)
W/S: + / - selected joint
R/F: + / - all joints (coarse)
O:   Open gripper
C:   Close gripper
X:   Reset robot to vertical (all-zero) position
ESC: Quit

Selected joint: crane_plus_joint1
```

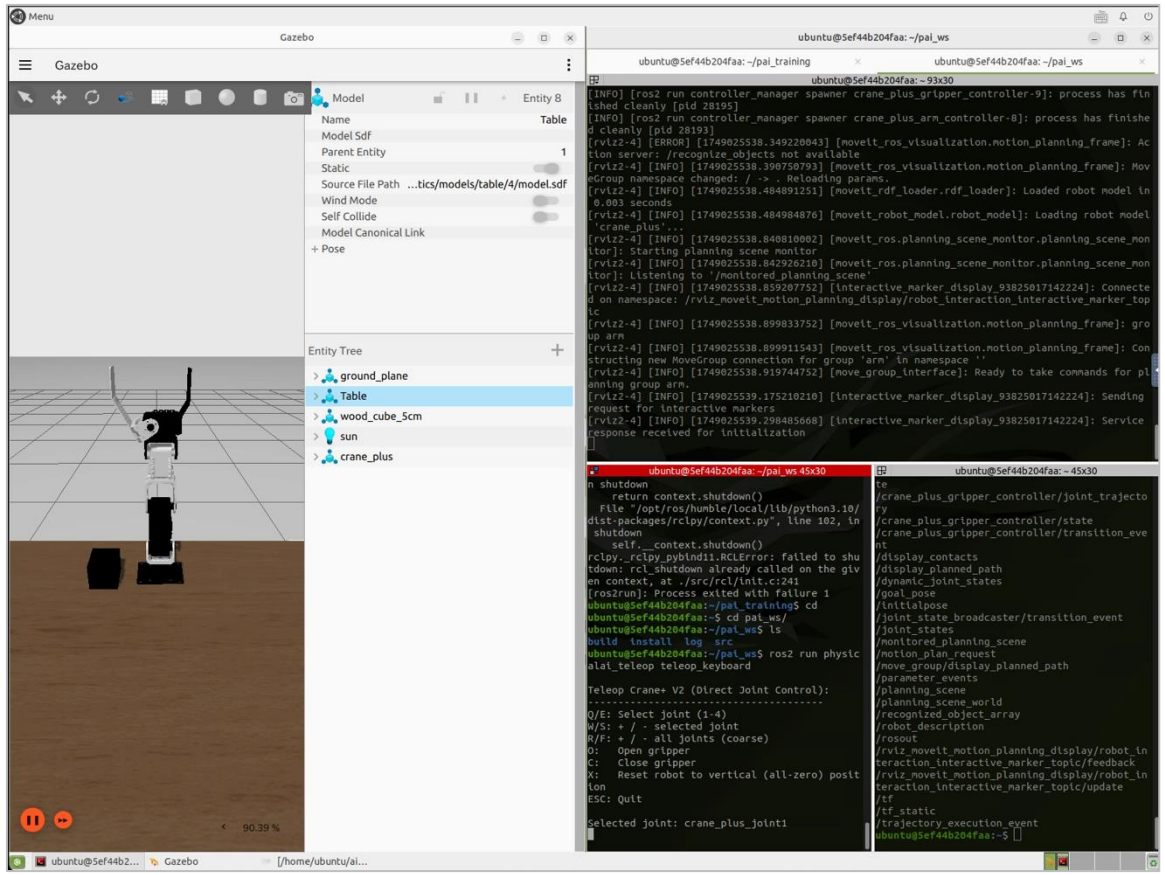
テレオペノード起動（別ターミナル）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

キー操作とテレオペ画面

- 色々押して遊んでみよう
- **x** で初期姿勢 [0, 0, 0, 0, 0] に戻る

Key	Function
Q / E	操作する関節を選択（起動時は joint1．Q 1回で joint3，E 1回で joint2）
W / S	選択した関節角を増加／減少
R / F	全関節角を増加／減少（粗調整）
O	グリッパを開く
C	グリッパを閉じる
X	直立姿勢（全関節0）に戻す
ESC	終了

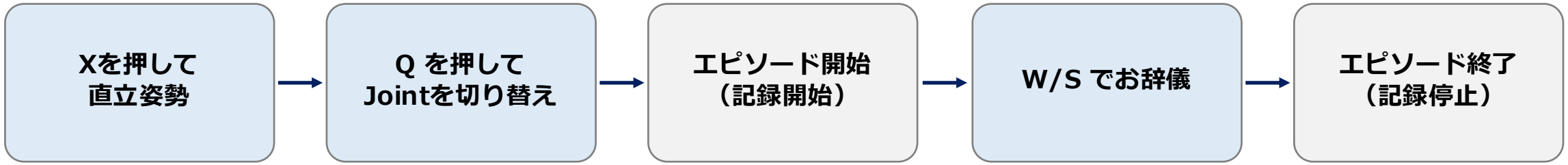


余力あれば、今回、関節空間でのテレオペは直感的でない
→ 手先空間テレオペへの改善も歓迎

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

データ収集：お辞儀（pekori）デモ

- ロボットが**お辞儀するようなデモを10～20エピソード**ほど集める
- **1エピソード ＝ 1 rosbag** データに対応



- Gazeboとテレオペのノードが立ち上がった状態で，**新たにターミナル**を立ち上げて記録する
- キー入力はテレオペノードのターミナル、記録の開始・停止はrosbagのターミナルで行う
  – どのターミナルにフォーカスが当たっているかに注意
- 2エピソード目以降は，直立姿勢から **W を数回押す**だけでお辞儀できる（関節は選択不要）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

データ収集：記録の手順

1エピソード目（ep0）

```
# Start recording
ros2 bag record -a -o ~/dataset/rosbags/pekori/ep0   # ep1, ep2, ... と増やしていく
```

- テレオペのターミナルにフォーカス → **q** を1回押して joint3 を選択（Selected joint: crane_plus_joint3 と表示されるか確認）→ **w** を数回押してお辞儀（グリッパも使うなら → C で閉じる）
- rosbagのターミナルにフォーカス → **Ctrl+C** で記録停止
  → テレオペのターミナルで **x** を1回押して直立姿勢へ

次のエピソード（ep1）

```
# Next episode
ros2 bag record -a -o ~/dataset/rosbags/pekori/ep1
```

- テレオペのターミナルで **w** を数回押してお辞儀 → rosbagのターミナルで **Ctrl+C** → **x** で直立姿勢へ
- これを**10〜20回**ほど繰り返す．終わったらテレオペノードは **Ctrl+C** でkillしてOK

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】データ収集のコツ：データの質が方策の質を決める

集めたデモがそのまま方策の手本になる．記録を始める前に，以下を意識する

注意点	理由	今回（お辞儀）では
初期状態から必ず始める	学習データにない状態（OOD）から始まるのを防ぐ	毎回 **x** で直立に戻してから記録
グリッパーはできる限り 0 / 1	状態遷移に支配的. 目標をはっきりさせ，スタックを防ぐ	今回はしないが、お辞儀に組み合わせても良い
停止を作らない	途中で止まると「止まる」ことを学習する	Subscribed 直後に動かし，終えたらすぐ **Ctrl+C**
軌道を一意にする	ACT等は多峰性（いろいろな軌道）が苦手 上から／横から，取る順番などを決める	**W** の回数をそろえる（例：10回＝0.5 rad）

※ 変換時，短いエピソードは最終フレームで埋めて最長にそろえられる → **長さのばらつきも「停止」を増やす**

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

rosbag2lerobot変換

- rosbagは**非同期のトピック通信を全て記録したもの**
  → 学習に使うトピックを選び，**学習に使いたい周期でサンプリング**する必要がある
- 今回はLeRobotで学習するため，rosbag形式のデータを **LeRobotDataset** 形式に変換する



補足：データセット形式

- LeRobotDatasetの他によく使われる形式：**RLDS**（特にOpen X-Embodimentなどの大規模データ）や **HDF5**
- LeRobotは画像データを**ビデオ（mp4）に圧縮**して保存するため，これらに比べて
  データセットサイズを大きく抑えられる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

rosbag2lerobot変換：変換の実行

- 変換には先ほど環境構築した **pai_training** を使う

```
cd ~/pai_training
```

- **rosbag2lerobot.py** による変換は pixi の task として設定してある（pyproject.toml の [tool.pixi.tasks]）

```
pixi task list      # タスク一覧（convert / train / infer / debug-policy）
pixi run convert    #= pixi run python scripts/rosbag2lerobot.py
```

- **configs/conversion_config.yaml** で以下を設定する
  - 入力：変換元のrosbagのディレクトリ
  - 出力：変換出力先
  - 周波数：ターゲット周波数（fps）
- データセット名（repo_id）・タスク名（task_name）・特徴量は **configs/crane_features.py**
- 学習の設定は **configs/train_config.yaml**

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

rosbag2lerobot変換：変換したデータの可視化

```
pixi run python scripts/visualize_dataset.py \
    --root /home/ubuntu/dataset/lerobot_dataset/crane_plus_pekori \
    --repo-id crane_plus_pekori --episode-index 1 --mode distant
```

- Firefoxが立ち上がったら，先ほどと同じ（URLを手元PCのブラウザに貼り付け）
- 自分で集めたrosbagデータをLeRobotDataset形式に変換し，**可視化できた**
  - action（指令）と state（関節状態）の対応を確認



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

学習：ACT（Action Chunking with Transformers）

- 変換したLeRobotDatasetで，**ACT** というTransformerベースの模倣学習モデルを学習する
  - 軽量でローカルのパソコンで動かしやすく，短時間で少数サンプルでもある程度の方策の学習可能

- 学習にも pai_training を使う．ログのため wandb にログイン（最初に wandb を更新．ログインなしでも可 → 次ページ）

```
cd ~/pai_training
pixi add --pypi "wandb>=0.22"    # 新しい形式のAPIキーに対応させる（最初の1回）
pixi run wandb login
# Access https://wandb.ai/authorize to get your API key. This can be done in your local PC.
```

- 学習を実行．**`configs/train_config.yaml`** に学習のパラメータが設定されている

```
pixi run train
```

今回は実際に動かすエンジニアリングにフォーカスし，モデルの詳細は省略

気になる方は以下でACTの詳細を学習しよう！

- 論文：https://arxiv.org/abs/2304.13705
- Hugging Face: https://huggingface.co/docs/lerobot/act

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】 wandb はログインあり／なしどちらでも

wandb（Weights & Biases）：学習の経過をグラフ表示・記録・比較できるWebサービス
使っても使わなくても，**学習と保存されるモデルは同じ**

	A. ログインあり	B. ログインなし
アカウント	必要（無料）	不要

コマンド	`pixi add --pypi "wandb>=0.22"`
	`pixi run wandb login`
	`pixi run train`
		`WANDB_MODE=offline pixi run train`

学習の経過	表示されるURLでグラフを確認	ターミナルに出る loss を確認
向いている場面	実験を何回も回して比べる・共有する	まず動かしてみる・当日アカウントがない

⚠ ログインする場合は，最初に `pixi add --pypi "wandb>=0.22"` で wandb を更新する
固定されている wandb 0.19.11 は新しい形式のAPIキー（wandb_v1_…，86文字）に対応しておらず，「API key must be 40 characters long」で止まるため

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】変換の処理詳細（1/2）

1. 初期化・設定読み込み

- 入力：rosbag2データ（複数エピソード），変換設定ファイル，出力先，目標サンプリング周波数（fps）
- 設定から特徴量（features），robot_type，repo_id，task_name などを取得
- **`LeRobotDataset.create`** で新しいデータセットを作成

2. エピソードの探索

- 指定ディレクトリ以下の全ての .db3 ファイル（rosbag2のデータベース）を探し，
  エピソードごとに処理

3. 各エピソードのメッセージ取得

- **`/joint_states`**（関節の現在状態）
- **`/crane_plus_arm_controller/joint_trajectory`**（アームへの指令）
- **`/crane_plus_gripper_controller/joint_trajectory`**（グリッパへの指令）
- 各トピックから全メッセージとタイムスタンプを抽出

[引用] matsuolab/pai_training（scripts/rosbag2lerobot.py）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

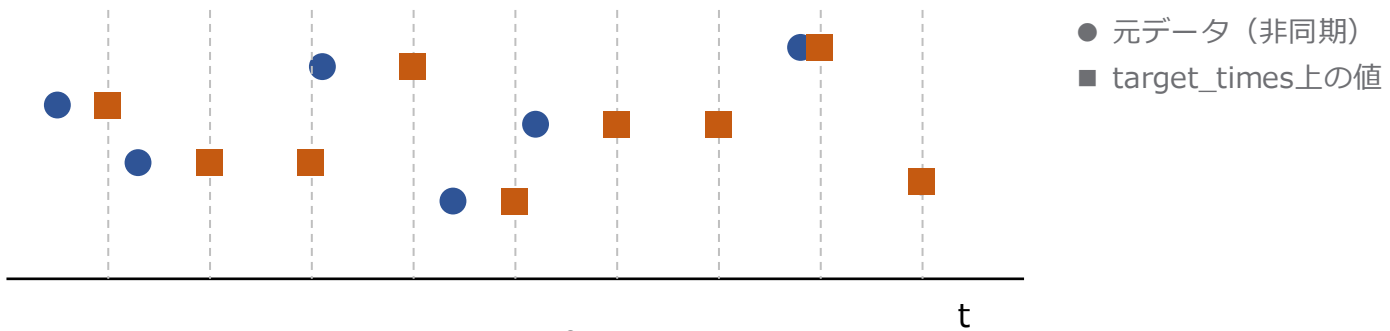
【補足】変換の処理詳細（2/2）：サンプリングと整形

4. サンプリングと整形

- 各トピックの共通の時間範囲（start_time～end_time）を決定
- 指定fpsで等間隔の **`target_times`** を生成
- 各時刻について**その時刻以前で一番近い値**を使う（直前値を保持）
- アームとグリッパの指令を結合して **action** とする

5. LeRobotDatasetへの追加（各フレーム）

- **`observation.state`**：サンプリングした関節状態 **`j_pos_sampled`**
- **`observation.environment_state`**：同上（同じ値）
- **`action`**：[アーム4軸 + グリッパ1軸] のベクトル
- **`task`**：タスク名　→　1エピソード分を **`dataset.save_episode()`** で保存



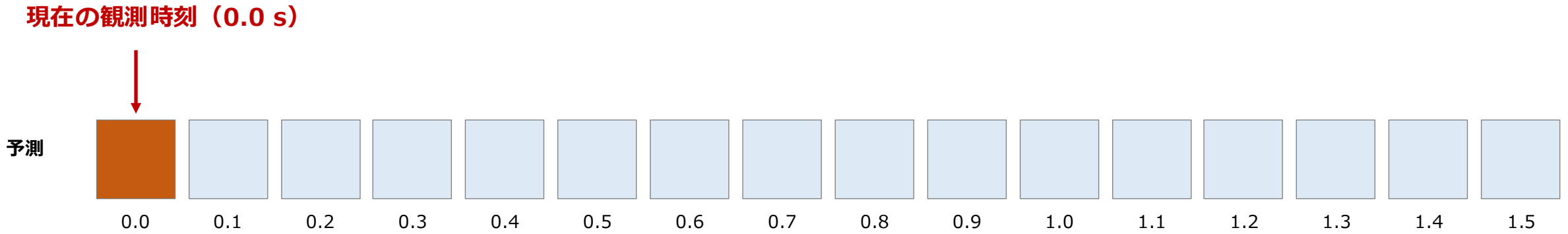
直前値保持によるリサンプリングのイメージ

[引用] matsuolab/pai_training（scripts/rosbag2lerobot.py）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

【補足】アクションチャンク

- 現在のロボット関節角を入力として，**現在から将来の関節角16ステップ分**を予測するように学習
- アクションチャンク：複数ステップの行動系列
  - **`action: [0.0, 0.1, …, 1.3, 1.4, 1.5]`**（観測時刻からの相対時刻，単位は秒）



アクションチャンクのイメージ

今回では

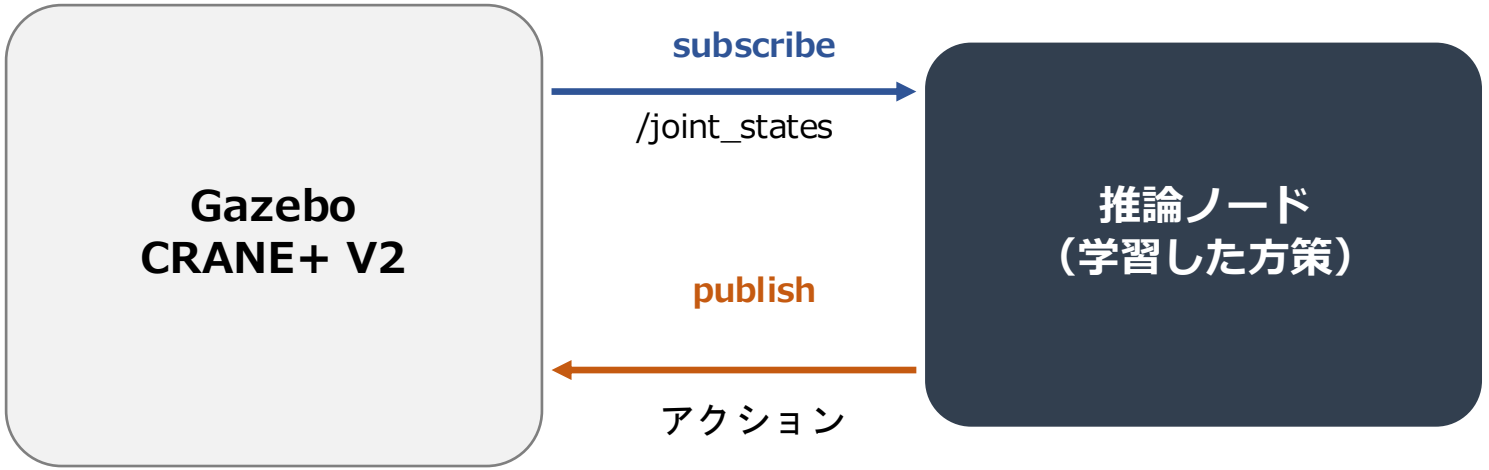
- 実際に実行（execute）されるのは**最初のアクションのみ**
  - **`n_action_steps: 1`**（train_config.yaml）
  - `configs/train_config.yaml` の `delta_timestamps.action` を `[0.0, 0.1, …, 1.5]` に書き換えてから学習（初期値は -0.1 始まり）
- 毎ステップ推論し直すことで，観測の変化にすぐ反応

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

デプロイ

- 推論用のノードを実行し，学習した方策をGazebo環境にデプロイする
- 実行時はコマンドライン引数にモデルへのパスを指定する

```
pixi run infer --model-path /home/ubuntu/checkpoints/train/crane_plus_pekori_act
```



- 現在の観測（**`/joint_states`**）をsubscribeして方策に与え，推論されたアクションを関節指令トピックにpublishする

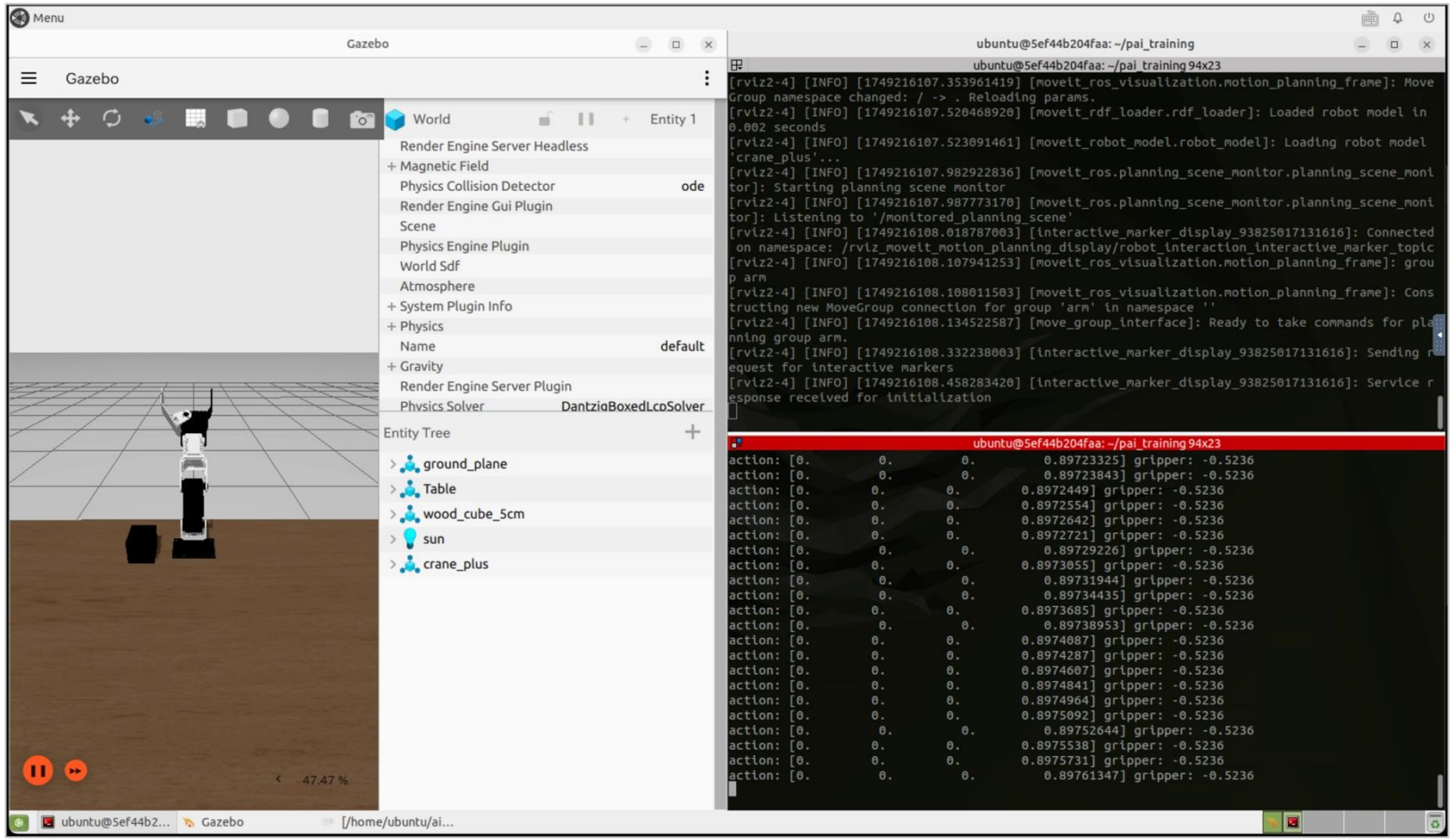
⚠ **テレオペノードは必ず止めておく：**キーを押さなくても 20Hz で指令を出し続けるので，方策の指令とぶつかる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

デプロイ結果

デプロイ達成

- Gazebo上でロボットがお辞儀する
- ターミナルに推論されたactionが表示される



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

方策のデバッグ

- シミュレータ（or 実機）でのデプロイにより，方策の性能を**定性的に評価**できる
- 性能が良くなかった場合，原因を切り分ける必要がある

よくある原因例：

- 学習に使っていないデプロイ時の観測に方策が汎化していない（今回は学習とほぼ同じ）
- そもそも学習データに対しても正しい行動を出せていない

→ 収集したデータセットの観測を方策に与え，推論と実際のアクション（Ground Truth, GT）を比較

```
pixi run debug-policy --dataset-root /home/ubuntu/dataset/lerobot_dataset/crane_plus_pekori \
    --repo-id crane_plus_pekori --policy-path /home/ubuntu/checkpoints/train/crane_plus_pekori_act \
    --episode-index 1
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

デバッグ結果の読み方

データセット上での比較	デプロイ時の挙動	考えられる原因・次の一手
予測がGTとずれている	うまく動かない	学習がうまくいっていない → データ量・品質，学習設定を見直す
予測がGTとよく一致	うまく動かない	デプロイ時の観測に汎化していない → 初期状態・観測の差を確認，データの多様性を増やす
予測がGTとよく一致	うまく動く	OK．エピソードや条件を変えて評価を続ける

まずは**学習データ上で方策が正しく振る舞えるか**を確認してから，汎化の問題を疑う

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

5. まとめ

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

まとめ

本講義では，ROSを使ったデータ収集を学び，ロボットデータの**収集・変換・学習・デプロイ**という一連の模倣学習の流れを体験した

1. **（復習）ROSの基本的な通信方式を理解**
2. **rosbag2を使用したデータ収集の方法を習得**
3. **LeRobotDataset形式への変換とデータの可視化を実践**
4. **収集したデータを用いた方策の学習**
5. **学習した方策のGazeboシミュレータ環境へのデプロイ**
6. **方策のデバッグと性能評価の手法を理解**

※ 解決が難しいerrorなどありましたらslackやgithubのissueで相談ください

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

もっと試してみよう：LeRobot などの公式ノートブック

LeRobot には ACT 以外にも多くの方策がある．**公式のノートブック・ドキュメント**から試そう

手法	種類	ノートブック／URL	
ACT	模倣学習	**ノートブック：**training-act.ipynb（huggingface/notebooks） https://github.com/huggingface/notebooks/blob/main/lerobot/training-act.ipynb https://huggingface.co/docs/lerobot/act	今回の演習 （自分のデータで学習済み）
Diffusion Policy	模倣学習	**ノートブック：**5_ros2_2_lerobot_diffusion.ipynb（講座の演習ノートブック） https://github.com/real-stanford/diffusion_policy https://diffusion-policy.cs.columbia.edu/ https://huggingface.co/lerobot/diffusion_pusht	Optional（PushT）
SmolVLA	VLA	**ノートブック：**training-smolvla.ipynb（huggingface/notebooks） https://github.com/huggingface/notebooks/blob/main/lerobot/training-smolvla.ipynb https://huggingface.co/docs/lerobot/smolvla	Optional
π0.5	VLA	**ノートブック：**なし（公式ドキュメントの手順で試す） https://huggingface.co/docs/lerobot/pi05 https://github.com/Physical-Intelligence/openpi	Optional（紹介のみ）
Isaac Lab × G1	強化学習	**ノートブック：**なし（NVIDIA 公式の入門コース） https://github.com/isaac-sim/IsaacLab https://github.com/unitreerobotics/unitree_rl_lab https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/ https://github.com/mujocolab/mjlab	Optional（GPU搭載PCが必要）

- 講師作成の SmolVLA などの資料を共有します
  - 共有ドライブ

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO



松尾・岩澤研究室

MATSUO-IWASAWA LAB UTOKYO

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO